TIBER CAD

Free Download

Release 2.0

TiberLAB srl

April 1, 2011

CONTENTS

1 Installation instructions **1**
- 1.1 Prerequisites . . . 1
- 1.2 Windows installation procedure . . . 1
- 1.3 Linux installation procedure . . . 1
- 1.4 Quick start guide . . . 2
- 1.5 Bug reports / Feedback . . . 2

2 Input for TiberCAD **3**
- 2.1 Description of Input file structure . . . 3
- 2.2 Device section . . . 4
- 2.3 Modules . . . 5
- 2.4 Simulation section . . . 8
- 2.5 Output description . . . 9
- 2.6 Example of Input file . . . 10

3 Elasticity **15**
- 3.1 Abstract . . . 15
- 3.2 Mini theoretical intro . . . 15
- 3.3 Example . . . 15

4 Drift Diffusion **17**
- 4.1 Step 1 - Modeling the device . . . 17
- 4.2 Step 2 - Meshing the device . . . 19
- 4.3 Step 3 - TiberCAD Input file . . . 19
- 4.4 Step 4 - Run TiberCAD . . . 21
- 4.5 Output . . . 21

5 Thermal **23**
- 5.1 Introduction . . . 23
- 5.2 Boundary conditions . . . 24
- 5.3 Heat sources . . . 24

6 Module efaschroedinger **27**
- 6.1 Module options . . . 27
- 6.2 Solver section . . . 27
- 6.3 Physics section . . . 28
- 6.4 Output . . . 28

7 Module opticskp **29**
- 7.1 Output . . . 29

8 Module opticalspectrum **31**
8.1 Output 32

9 Module quantumdispersion **33**
9.1 Output 33

10 Module quantumdensity **35**
10.1 Output 35

11 Simulation Dye Solar Cells **37**
11.1 Model DSC 37
11.2 Boundary Conditions 37
11.3 Generation model 38

12 Glossary **43**

Bibliography **45**

Index **47**

CHAPTER

ONE

INSTALLATION INSTRUCTIONS

In the following, VERSION denotes the version number of the TIBERCAD release you downloaded and INSTALLPATH denotes the directory where TIBERCAD gets installed. Version 2.3.0 of **GMSH** (http://www.geuz.org/gmsh) will be installed together with TIBERCAD. For the Linux version of GMSH you need OpenGL libraries installed on your system.

1.1 Prerequisites

Get the installer package for your OS/architecture from http://www.tibercad.org or by contacting support@tibercad.org. Table 1 lists the packages available for download. To run TIBERCAD you will also need a license file that you will have to copy into the installation directory of TIBERCAD. In the Windows version, some graphical features such as graphical convergence monitors are only available if an X Window server is installed and running.

1.2 Windows installation procedure

To install TIBERCAD in Windows, run the setup program `tibercad-version_setup.exe` .

During the installation you can choose the installation directory. After finishing installation, copy your license file *(tibercad.lic)* into the

license subdirectory of the TIBERCAD

installation directory (INSTALLPATH/license), without changing its filename.

installer package name	*Target architecture*
tibercad-version_setup.exe	Windows 32-bit
tibercad-version_installer.bin	Linux 32-bit self-extracting installer

Table 1: Installer packages

1.3 Linux installation procedure

To install TIBERCAD in Linux, download and run the self-extracting installer `tibercad-version_installer.bin` and follow the installation instructions.

After installation, copy your license file *(tibercad.lic)* into the ?license? subdirectory of the TIBERCAD installation directory (INSTALLPATH/license) without changing the filename. You can also provide the license file during installation. The standard method to launch TIBERCAD is by means of a shell script that is installed alongside the TiberCAD executable. It takes care of setting all necessary environment variables. If for some reason you have to run the executable directly, remember to set TIBERCADROOT to the TiberCAD installation directory (INSTALLPATH).

1.4 Quick start guide

In the ?examples? subdirectory you can find several examples ready to run. They are the same as the tutorials on http://www.tibercad.org/documentation/tutorial/list.

1.4.1 Windows

Open Windows Explorer and go to the TIBERCAD installation directory. If you have write permission in the installation directory, you can browse to an examples directory and start the simulation by double clicking the input file, e.g. bulk.tib in Example 0. If not, copy the whole directory to a location in your personal area and run the examples from there. If you cannot run TIBERCAD by double clicking an input file *(*.tib)* , then the input files are probably not correctly associated with the TIBERCAD executable. In this case, try to establish the association by right-clicking the input file, choosing `open with...`

`>> Choose Program...` `>> Browse...` , browsing to the TIBERCAD installation directory

and choosing the TIBERCAD executable, tibercad.exe. A directory containing the simulation results will be created with the name provided in the input file.

1.4.2 Linux

After the correct installation of TIBERCAD you should be able to run TIBERCAD from the command line using the command tibercad. If not, you probably have to add the bin subdirectory of the TIBERCAD installation directory to your PATH environment variable or start the TIBERCAD executable using the absolute path (INSTALLPATH/bin/tibercad). Copy the directory of the example you want to run, e.g. bulk Si, to your home directory or any place you have write permissions for. Change to the newly created directory and run TIBERCAD by (assuming Example_0)

```
$ tibercad bulk.tib
```

A directory containing the simulation results will be created with the name provided in the input file.

1.5 Bug reports / Feedback

Please send bug reports, feedback or suggestions to support@tibercad.org. When submitting bug reports, please always include the full version number of TIBERCAD you are running. The full version number appears in the first line of output when running the program:

$ tibercad

TiberCAD version 1.0.0-961

Usage: `tibercad` <inputfile>

CHAPTER

TWO

INPUT FOR TIBERCAD

Input for TIBERCAD is composed by an input file e.g. “input.tib” and a mesh file generated by a mesher software: as for now, mesh files from GMSH (*.msh, v.1 and v.2.0) and from ISE-TCAD (*.grd) are supported. Be sure that the material files are in the correct directory . To run the program, type: **tibercad** *input_file_name*

2.1 Description of Input file structure

A valid input file for TIBERCAD is a text file with the structure described in the following. In the whole input file, everything following a ?#? is considered as a comment and is disregarded; blank lines can be present anywhere and are disregarded too. Input file is composed by several **blocks** : A block is enclosed between **“{“** and **“}”** brackets and may include one or more blocks. Each block has a header made of one or two keywords. Each block may contain zero or any number of **parameter assignements** in the form:

” tagname = tagvalue” , where

- *“tagname”* is a string
- *“tagvalue”* is a single numerical or string item or a list of items between “(” and ”)”

parenthesis and separated by commas. e.g. (*cathode, anode*)

Format is free for the parameter assignements, provided that they are separated by spaces. Everything which follows a ?#? is considered as a comment and is disregarded. For example:

```
electron_mobility field_dependent
  {
   region = buffer
   # type = doping_dependent
   low_field_model = constant
  }
```

Here and in the whole input file a string item can include a combination of characters, special characters and numbers, except spaces; if a space is found, the string item is taken as terminated.

The input file is composed by the following main classes of blocks:

Device, Module, Simulation

which will be described in the following.

2.2 Device section

```
Device hemt1
  {
   meshfile = hemt_msh.grd
   Region buffer
     {
      .........
     Doping
       {
        .........
       }
     }
   Region barrier_1
     {
      ............
     }
  ..................
  }
```

Device section includes the geometrical desription of the device to be simulated; an optional *device name* can be associated to the Device object, e.g. *Device hemt1* .

In Device section, two kinds of block can be present: the Region block and the Cluster block. Outside of these blocks, general options common to all the device can

be defined. The most important is the mesh file definition:

meshfile : name of mesh file. N.B.: the extension is mandatory! (*.grd* for ISE-TCAD,

.msh for GMSH mesh file v.1 and v.2.0)

The definition of the mesh file associated to this simulation is compulsory:

```
meshfile = hemt_msh.grd
```

The **Region** blocks contain the description of the device in continuous media ap proach; the **Cluster** blocks define each a group of regions (mesh regions) even with different physical properties, but to be treated together somewhere in the simulation (e.g. quantum calculation). In this way it is possible to refer to the set of these regions simply by the **Cluster** name.

Each **Region** block must be preceded by the keyword **"Region"** , followed by the (single-word) name of the **TiberCAD Region** . The name of the **TiberCAD Region** can coincide with the name of a mesh region, as defined during the modeling of the device; in this case, if the keyword *mesh_regions* is absent, the **TiberCAD Region** will be associated to the mesh region identified by the name assigned to the **TiberCAD Region** . Otherwise, the **TiberCAD Region** will be associated to the mesh regions specified by the keyword *mesh_regions* .

```
Region QWell
  {
   mesh_regions = (weel1,well2)
   structure = wz
   y-growth-direction = (1,0,-1,0)
   z-growth-direction = (-1,2,-1,0)
   x-growth-direction = (0,0,0,1)
   material = GaN
   Doping
     {
      density = 1e17
      type = donor
      level = 0.025
```

```
    }
  }
```

Here are the description of the available keywords for a **Region** block.

material (mandatory): name of the material associated to the present region, e.g Si;

it may be a ternary alloy, e.g AlGaAs, in this case keyword x described in the following has to be present.

x : alloy concentration, expressed as the molar fraction of the first component of the alloy; e.g. to express an alloy $Al_xGa_{1-x}As$ with molar fraction x = 0.2, that is $Al_{0.2}Ga_{0.8}As$, we select **AlGaAs** for the keyword material, and 0.2 for the keyword x, thus we write x = 0.2.

`mesh_regions` : (a list of) region physical name(s) as specified in the meshing program.

`structure` : crystal structure (wz = wurtzite, zb = zincblend)

`x-growth-direction, y-growth-direction, z-growth-direction` : Bravais vectors with Miller

indexes for wurtzite crystal (4 element vectors) or zincblende crystal (3 element vectors).


A common crystal structure and growth directions definition may be applied to all the Regions of the Device, just by defining them everywhere in the Device block, but outside any specific Region block.

Subblock doping, with the keywords:

density : doping concentration [cm-3] type : donor or acceptor level : energy level of the dopant [eV]

```
Doping
  {
   density = 1e21
   type = donor
   level = 0.026
  }
```

Each **Cluster** block must be preceded by the keyword **"Cluster"** , followed by the (single-word) name of the **Cluster** .

```
Cluster Quantum_1
  {
   regions = (well, barrier1,barrier2)
  }
```

regions (mandatory): list of physical regions as specified in the meshing program, or TIBERCAD region names to be grouped in the cluster.

Regions and **Clusters** represent the macroscopical description of the device or struc

ture to be be simulated in **TiberCAD** . In the rest of the input file, the physical regions associated to the **Modules** will be indicated by means of the **TiberCAD Region** and

Cluster names.

2.3 Modules

```
Module driftdiffusion
  {
   name = dd
```

```
   #regions = all
   statistics = FermiDirac
   strain_simulation = macrostrain
   plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
   eCurrentDensity, hCurrentDensity,CurrentDensity, ContactCurrents,
   ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)
    ..........
   Solver
     {
      nonlinear_solver = tiber
      nonlin_max_it = 15
      nonlin_rel_tol = 1e-12
      #nonlin_abs_tol = 1e-15
      nonlin_step_tol = 1e-5
     }
   Physics
     {
      polarization (piezo, pyro) {}
      mobility
        {
         type = field_dependent
         low_field_model = doping_dependent
        }
     }
Contact cathode
  {
   voltage = $Vd
  }
Contact anode
  {
   voltage = 0.0
  }
}
```

One or more module-blocks may be present: each module-block must be preceded by the keyword **"Module"** , followed by the (single-word) module name. This must be the name of one of the TIBERCAD modules. Here are the Modules implemented until now:

- `driftdiffusion` : Poisson-driftdiffusion transport of electrons and holes
- `thermal` : Heat balance simulation
- `excitontransport` : Exciton transport model
- `macrostrain` : Calculation of Elastic deformations in heterostructures
- `efaschroedinger` : Envelop Function Approximation (EFA) solution of single particle

Schroedinger equation for electrons and holes

- `quantumdensity` : Calculation of quantum density of electrons and holes.
- `quantumdispersion` : Dispersion of quantized states in k space
- `opticskp` : Optical properties (optical kp matrix elements)
- `opticalspectrum` : Emission spectrum (with k-space integration)
- `Sweep` : Parameterized execution of a module simulation (e.g. for the calculation

of output current characteristics)

- `Selfconsistent` : coupled calculations of different simulation modules.

- `DSC` : Simulation of a DSC solar cell1.

Each module-block usually contains a list of general options, such as plot and others specific to each module. Then, two main blocks define the Physics and the Solver models and parameters for this module.

Solver contains the solver parameters; depending on the Module, it can contain a

LinearSolver definition subblock.

Physics usually contains the definition of the physical models used in the simulation.

For example, for driftdiffusion module, recombination, electron mobility, trap, polarization and so on. A particular model is the Boundary model, which has an alias Contact for driftdiffusion module. The declaration of these models obey to the following syntax:

model_keyword type_specifier <block>

where *model_keyword* is the name of the physical model to be declared (e.g. trap model) and **type_specifier** is the name of a particular one among the available descriptions for that model. For example, for a trap model of **type** *acceptor*

```
trap acceptor
  {
   region = buffer
   Nt = 7e16
   Et = 0.5
   reference = cb
  }
```

An alternative multiple declaration is possible if no parameters, beyond default, are declared:

model_keyword (type specifier1, type specifier2,...)

```
recombination (srh, direct, auger) { }
```

In this example, several recombination models are defined (srh, auger, direct) each one with default parameters. For a detailed description of the models, please refer to the reference guide. Two special Modules are: the Sweep Module and the Selfconsistent Module

2.3.1 Module sweep

```
Module sweep
  {
   name = sweep_drain
   solve = driftdiffusion
   variable = $Vd
   start = 0.0
   stop = 2.0
   steps = 20
   plot_data = true
  }
Module sweep
  {
   name = sweep_gate
   solve = sweep_drain
   variable = $Vg
   start = -0.1
   stop = 1.0
   steps = 11
  }
```

Each sweep Module defines a set of calculations applied to a boundary region (e.g. a set of bias values to be assigned to a drain contact of a MOSFET for the calculation of an output drain IV characteristic), in this Guide referred to as sweep calculation.

The following keywords are defined for this feature:

`variable` : name of the variable to which the sweep is applied:

E.g.: variable = $Vg

indicates that the values are applied to the variable Vg, which is a quantity defined in a Contact definition

`start, stop, steps` : sweep starts from start value, is repeated steps times and stops

in stop

`solve` : name of the simulation (module) associated to the sweep calculation; it may

be the name of another sweep defined in the same block.

`plotvariable` (obsolete): specify the integrated quantity to be calculated during the

sweep and that will be shown in the output file sweep modelname sweepvariable.dat, eg. sweep driftdifiusion Vb.dat for a sweep of current calculation on the variable Vb (typically a contact voltage).

`plot_data` : default is false; if it is set to true, then output data will be written for

each step of the sweep calculation, otherwise just the results for the final step will be present in the output.

Once a *sweep* calculation has been defined, it is treated as a special case of simulation and may be executed as an usual simulation: by adding it in the solve list, e.g. `solve = sweep_drain` .

2.3.2 Module selfconsistent

In this Module it is possible to define a self-consistent calculation based on two different simulation modules (e.g. driftdiffusion and excitontransport).

```
Module selfconsistent
  {
   name = sc_all
   solve = (tb, dens_el, dens_hl, dd)
   max_iterations = 5
   abs_tolerance = 1e-3
   rel_tolerance = 1e-6
  }
```

In **solve** the list of simulations to be performed self-consistently is specified. Now it is possible to execute the specified simulations in self consistent way, by using the **selfconsistent** keyword like a simulation name, in any **solve** assignement, e.g. solve = selfconsistent in **Simulation** section, or even in a **sweep** section.

2.4 Simulation section

In this block one can specify several general parameters and settings for the actual calculation to be run, such as the temperature, the process-flow of simulation, etc.

`searchpath` : path for material files

`mesh units` : units of measurements used in the meshing (relative to meters): e.g.,

10^{-6} for μm

`dimension` : dimension of simulation (1,2,3)

`temperature` : temperature of the system [K]

`solve` : list of simulations to be executed, in the order of execution; if the list contains

"sweep", a sweep is performed as specified in sweep block in the Solver section.

```
solve = (strain,driftdiffusion, quantum_electrons,
quantum_holes)
```

`resultpath` : path for output directory

`output format` : format of the output data: *gmv* for **GMV** , ise for **Tecplot** , grace for

xmgr (ascii data column type), *vtk* for **Paraview** .

`plot` : list of output variables which are calculated and available in output files. It

may be overriden by defining list specific to one or more modules.

2.5 Output description

At the end of the execution, the program will write the results of the simulation in the directory specified by resultpath , with the format specified by output format. The output variables are specified in a list plot, in each Module.

TiberCAD output is divided in two classes: **mesh-based** and **mesh-independent** quantities.

Mesh-based quantities are all the quantities associated with the nodes of the mesh, such as Fermi level, electron and hole density, conduction and valence band, etc., together with all the quantities associated with the elements of the mesh, such as current density.

The output values for these quantities are reported in the files simname msh.ext, where *simname* is the simulation module used for the calculations and ext is the extension of the chosen file format, e.g. vtu for paraview output.

```
strain_msh.vtu
```

In the case a sweep calculation is performed and the plot data keyword is set to true, the output files are of the kind simname sweepvariable step value msh.ext, where

sweepvariable is the variable with respect to which the sweep is performed (e.g. gate

voltage) and step value is the value of this variable at that step; e.g the result at the step *Vbias = 1.1* will be found in the file:

```
dd_Vbias_1.1_msh.vtu
```

`mesh-independent` quantities are the quantities which are not associated to the

mesh, for example current at the contacts of a diode or quantized energy levels in a quantum well. These **mesh-independent** quantities are displayed in separated files, with the format simname.ext, e.g `quantum_electrons.dat` , where simname is the name of the model (simulation) associated to the results. If a sweep is performed, the output file gets the format sweep name simname.ext, where *sweep_name* is the name of the sweep performed, for example

```
sweep_drain_driftdiffusion.dat
```

Inside the file, output values for all the steps of calculation are shown.

2.6 Example of Input file

Here is an example of the input file template:

```
Device hemt1
  {
   meshfile = hemt_msh.grd
   mesh_units = 1e-6
   material = GaN
   y-growth-direction = (0,0,0,1)
   z-growth-direction = (1,0,-1,0)
   x-growth-direction = (-1,2,-1,0)
   Region barrier
     {
      material = AlGaN
      x = 0.14
     }
   Region barrier_doped
     {
      mesh_regions = (barrier_dop_s, barrier_dop_d)
      material = AlGaN
      x = 0.14
      Doping
                {
         density = 1e21
         type = donor
         level = 0.026
        }
     }
  Region buffer { }

      Region buffer_doped
    {
     mesh_regions = (buffer_dop_s, buffer_dop_d)
     Doping
           {
        density = 1e21
        type = donor
        level = 0.026
       }
    }
  Region cap
    {
     Doping
           {
        density = 5e18
        type = donor
        level = 0.026
       }
    }
  Region cap_doped
    {
     mesh_regions = (cap_dop_s, cap_dop_d)
     Doping
           {
        density = 1e21
        type = donor
        level = 0.026
```

```
      }
    }
  Region passivation
    {
     mesh_regions = (passiv1, passiv2, passiv3, passiv4)
     material = SiN
    }
  Cluster semiconductor
    {
     regions = -passivation
    }
  }
Module driftdiffusion
  {
   name = dd
   regions = all
   coupling = electrons
   save_state = true
   #load_state = output_Nt7e16_out/dd_Vd_50.tsv
   statistics = FermiDirac
   strain_simulation = macrostrain
   plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
           eCurrentDensity, hCurrentDensity,CurrentDensity, ContactCurrents,
           ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity,)

       NonlinearSolver linesearch
     {
      # type = ls
      relative_tolerance = 1e-15
      step_tolerance = 5e-3
      max_iterations = 15
      LinearSolver petsc
        {
         ksp_type = pconly
         preconditioner = lu
        }
     }
   Physics
     {
      recombination (srh, direct, auger) { }
      electron_mobility field_dependent
        {
         region = buffer
         low_field_model = constant
        }
      electron_mobility doping_dependent
        {
         region = (barrier, barrier_doped, cap, cap_doped, buffer_doped)
        }
      trap fixed_charge
        {
         region = GaN/SiN
         Nt = 2.74e13
        }
      trap acceptor
        {
         region = buffer
         Nt = 7e16
```

```
         Et = 0.5
         reference = cb
        }
     polarization (piezo, pyro) { }
    }

  Contact drain
    {
     voltage = @Vd
    }
  Contact source
    {
     voltage = 0.0
    }
  Contact gate
    {
     regions = (gate1, gate2)
     type = schottky
     work_function = 1.5272
     voltage = @Vg
    }
  }

Module macrostrain
  {
   regions = -passivation # all the device except Region "passivation"
   plot = all
   LinearSolver # default
     {
      tolerance = 1e-7
      max_iterations = 10000
      #xmonitor = true
      pc = composite
     }
   Physics
     {
      Boundary substrate
        {
         regions = bottom
         material = GaN
         y-growth-direction = (0,0,0,1)
         z-growth-direction = (1,0,-1,0)
         x-growth-direction = (-1,2,-1,0)
        }

      Boundary extended_material
        {
         regions = side
        }
     }
  }

Module sweep
  {
   name = sweep_g
   variable = Vg
   solve = sweep_d
   start = -1
```

```
   stop = 1.0
   steps = 2
   max_step = 0.25
   min_step = 1e-4
   #plot_data = true
  }

Module sweep
  {
   name = sweep_d
   variable = Vd
   solve = driftdiffusion
   start = 0.0
   stop = 50.0
   steps = 200
   min_step = 1e-4
   plot_data = true
  }

Module selfconsistent
  {
   name = relaxation
   solve = (macrostrain, driftdiffusion)
   absolute_tolerance = 1e-3
   relative_tolerance = 1e-5
  }

Simulation
  {
   temperature = 300
   verbose = 3
   solve = (macrostrain, dd, sweep_g)
   logfile = output_Nt5e16_Al0.215_SiN/hemt.log
   # these parameters can be defined in modules, too
   resultpath = output_Nt5e16_Al0.215_SiN
   output_format = vtk
  }
```

CHAPTER

THREE

ELASTICITY

3.1 Abstract

Elasticity is a Finite Elements solver for mechanical equilibrium problems. It brings features typically developed for structural stability solvers into device modeling. The coupled treatment of the electro-mechanical problem within a unique framework results very useful to explore for multidisciplinary ideas.

3.2 Mini theoretical intro

Assuming a small displacement, Elasticity elasticity computed the strain by solving the equation

$$\frac{\partial}{\partial x_j} C_{ijlk} \frac{\partial u_l}{\partial x_k} = f_i$$

where C is the stiffness constant and f is the total external body force. The latter may include several contribution such as the converse piezoelectric effect the thermal expansion and a lattice mismatch induced strain. The latter, described in the example below, occurs when an interface is created between two materials with different lattice constants. Let ϵ^{LM} be the lattice mismatch strain, then the effective body force reads as :

$$f_i = -\frac{\partial}{\partial x_j} C_{ijlk} \epsilon^{LM}_{lk}$$

3.3 Example

In the following we will compute the strain induced by the lattice mismatch in a system comprising a layer of GaN between two contacts of $Al_xGa_{1-x}N$. Once the mesh is created with gmsh (distributed along TiberCAD) we may start to build the input file. First, we have to insert the region section

```
Device
  {
   dimension = 3
   meshfile = mesh.msh
   mesh_units = 1e-9
   material = AlGaN
   x = 0.2
   x-growth-direction = (-1,0,1,0)
   y-growth-direction = (-1,2,-1,0)
   z-growth-direction = (0,0,0,1)
```

```
   Region Well  {material = GaN}
  }
```

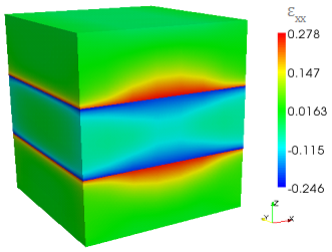
All the options included in this section, such ad the material, axis growth and alloy concentration, apply for the whole structure. If we want to specify a region with different properties, we may use the keyword Region and refer to a region name, as specified in the mesh file.

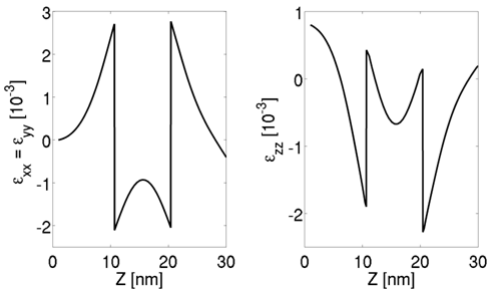
The second part of the input file is devoted to the module declaration

```
Module elasticity
  {
   Physics
     {
      BodyForcelattice_mismatch
        {
         x-growth-direction = (-1,0,1,0)
         y-growth-direction = (-1,2,-1,0)
         z-growth-direction = (0,0,0,1)
         reference_material = AlGaN
         x = 0.2
        }
      Contact Base {type = clamp}
     }
  }
```

In physics we declare the physical models to be applied to our device. In our case, with BodyForce lattice mismatch we are adding a body force into the system, induced by the lattice mismatch with the reference lattice which in this case is $Al_{0.2}Ga_{0.8}N$. In order to avoid a free standing device we may want to freeze a surface. With Boundary Clamp we fix, in this case, the region Base.

The third part is simply Simulation **{solve = elasticity}** where the default name *elasticity* is used. By default, files for 3D system are written in vtu which can be read from Paraview. Output data about strain are shown below.





CHAPTER

FOUR

DRIFT DIFFUSION

In this example we will see a very simple TiberCAD simulation:

1D calculation of Poisson and drift-diffusion for a bulk Silicon sample.

The following files should be in your working directory:

bulk.tib : **TiberCAD** input file

bulk.msh : mesh file

The mesh file can be obtained from the following GMSH geo file : bulk.geo

This is the summary of the Sections of this Tutorial :

- *Step 1 - Modeling the device*
- *Step 2 - Meshing the device*
- *Step 3 - TiberCAD Input file*
- *Step 4 - Run TiberCAD*
- *Output*

4.1 Step 1 - Modeling the device

As a first step, we have to model the device. To do so, you can use DEVISE module of ISE-TCAD 9.5 software package or GMSH program.

Here we'll see in details the procedure for GMSH.

There are two possible ways to use GMSH:

Interactive, using the graphical interface Using a script file In the following we'll see how to write a basic GMSH script (bulk.geo); for any details please refer to GMSH manual GMSH (http://geuz.org/gmsh/).

: In a **1D** simulation it is assumed that the geometrical model is restricted to the **x axis** . In a **2D** simulation it is assumed that the geometrical model is restricted to the **xy-plane (z=0)** Any other geometrical orientation could give impredictable results

In a GMSH script, several variables can be defined and given a value in this way:

```
L = 1
d = 0.01
```

these are valid GMSH variables: L is just the length of the Si sample; d is the value of a *characteristic mesh length* (see below).

Definition of geometrical entities **Points**

```
Point(1) = {0, 0, 0, d}
Point(2) = {L, 0, 0, d}
```

The first three expressions inside the braces on the right hand side give the three X, Y and Z **coordinates** of the point; the last expression **(d)** sets the **characteristic mesh length** at that point, that is the *"size"* of a mesh element, defined as the length of the segment for a line segment, the radius of the circumscribed circle for a triangle and the radius of the circumscribed sphere for a tetrahedron.

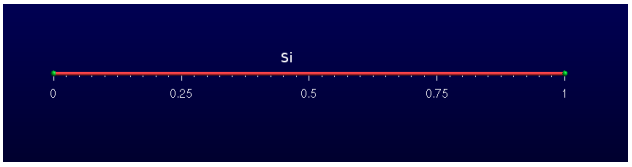
Thus, the smaller is the value of d, the greater is the mesh density close to that point.

The size of the mesh elements will then be computed in GMSH by linearly interpolating these characteristic lengths in the whole mesh.

Definition of a geometrical entity **Line**

```
Line(1) = {1, 2}
```

The two expressions inside the braces on the right hand side give the identification numbers of the start and end points of the line.



Definition of the physical entity **Physical Line bulk**

```
Physical Line("bulk") = {1}
```

The expression(s) inside the braces on the right hand side give the identification numbers of all the **geometrical lines** that need to be grouped inside the *physical line* .

In this way, in general, physical regions are created which associate together geometrical regions, and then the related mesh elements, which share some common physical properties. It's only these physical regions which can be referred to outside GMSH. In TiberCAD, this is done by associating one or more physical regions to a **TiberCAD region** through the keyword *mesh_regions* (see in the following).

Definition of two physical entities Physical Point:

```
Physical Point("anode2")   = {1}
Physical Point("cathode") = {2}
```

```
|warn| : In general, in a nD simulation, **(n-1)D** physical regions (points in 1D, lines in 2D, surf
```

```
are used by TiberCAD to impose the required boundary conditions.

 Each (n-1)D physical region defined in this way in GMSH will be associated in TiberCAD to a boundary
 through the keyword **BC_reg_numb** . Thus, in this case, Physical points Anode and Cathode will be
 to two **Contact** regions (see in the following).
```

4.2 Step 2 - Meshing the device

The *.geo* script file with the geometrical description can be run in GMSH, to display the modelled device and to mesh it through the GMSH graphical interface. Alternatively, a *non-interactive* mode is also available in GMSH, without graphical user interface.

For example, to mesh this 1D tutorial in non-interactive mode, just type:


gmsh bulk.geo -1 -o bulk.msh

where bulk.geo is the geometrical description of the device with GMSH syntax:

-1 means *1D mesh generation*

some command line options are:

-1 , -2, -3 to perform *1D, 2D or 3D* mesh generation,

-o **mesh_file.msh** to specify the name of the mesh file to be generated

In this way, a .msh has been generated and is ready to be read in TiberCAD.

4.3 Step 3 - TiberCAD Input file

Now we have to write down the **TiberCAD input file** (bulk.tib). For a detailed reference see the user guide (Input file) and Getting started 1

4.3.1 Description of Device Regions

First, we have to list all the **TiberCAD Regions** present in our Device: a TiberCAD Region is usually a section of the device featuring the same material and possibly the same doping.

```
Device
  {
   meshfile = bulk.msh

   Region bulk
     {
      material = Si

      Doping
        {
         Nd = 1e16
         type = donor
        }
     }
  }
```

The TiberCAD Region bulk is made of Silicon and n-doped with a concentration $1x10^{16}cm^{-3}$.

Through the keyword **Region** , one GMSH physical region (Physical Lines in 1D, Physical Surfaces in 2D, Physical Volumes in 3D) previously defined in the GMSH mesh (*Step 1 - Modeling the device*), can be associated to the present TiberCAD Region, in this way:

```
Region  GMSH_physical_region_name
```

In this case, the Physical Line bulk is associated to the TiberCAD Region bulk.

Alternatively, through the optional keyword **mesh_regions** , one or more GMSH physical regions can be associated to a single TiberCAD Region .

4.3.2 Definition of Simulation

Now we define the **Simulation** driftdiffusion_1: it belongs to the class **driftdiffusion**

```
Module driftdiffusion
  {
   name = driftdiffusion  # this is the default name

   regions = all          # 'all' is the default

   # what we want to plot
   plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
   ....
```

The TiberCAD simulation *driftdiffusion_1* , belonging to the model driftdiffusion, will be applied to the whole device structure (`regions = all`)

4.3.3 Definition of Boundary Conditions

The anode and cathode contacts of our 1D Si sample are defined as **Boundary conditions regions** (`Contact anode, Contact cathode`) in the following way:

Module driftdiffusion {

name = driftdiffusion

regions = all

plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)

Contact anode { voltage = $Vb } Contact cathode { }

}

Through the keyword Contact , one (n-1) -dimension GMSH physical region (Physical Point in 1D, Physical Line in 2D, Physical Surface in 3D) previously defined in the GMSH mesh (*Step 1 - Modeling the device*), can be associated to the present TiberCAD Contact, in this way:

Contact GMSH_physical_region_name

In this case, the *Physical Point* **anode** is associated to the TiberCAD Contact anode and the Physical Point cathode is associated to the TiberCAD Contact cathode.

Both contacts are defined as *ohmic* , cathod is assigned a fixed `voltage = 0.0` , while anode voltage is given by the value of the variable *Vb*

```
voltage = @Vb
```

4.3.4 Definition of Simulation parameters

Vb is specified in the sweep block, in the Solver section

Module sweep {

    solve = driftdiffusion variable = $Vb start = 0.0 stop = 1 steps = 10 plot_data = true

}

In this way, the simulation driftdiffusion_1 is performed for 10 (steps = 10) values of the anode voltage (variable = Vb), between 0 and 1.

For each step we want to plot the solution variables specified in the driftdiffusion module (plot_data = true).

4.3.5 Definition of Execution parameters

In the **Simulation** section , we decide *which* simulations to perform and in which *order*; set `solve = sweep` , to execute the sweep which run driftdiffusion_1 simulation for the specified loop.

```
Simulation
  {
   verbose = 2

   solve = sweep

   resultpath = output
   output_format = grace
  }
```

Output files with conduction and valence band profiles (plot = Ec,Ev..) and all the calculated values of the current at the contacts (*ContactCurrents*) (the IV characteristic) are generated.

To increases the amount of information written to the screen we can vary the verbose level (verbose = 2).

4.4 Step 4 - Run TiberCAD

Now we can run TiberCAD

by double clicking on bulk.tib file (in Windows)

or by command line in linux: tibercad bulk.tib

4.5 Output

The generated Output files are:

- driftdiffusion_materials.dat : material (mesh) regions, in this case just region 1
- driftdiffusion_nodal.dat : nodal quantities (here conduction and valence band)

- sweep_driftdiffusion_Vb.dat : integrated current at the two contacts for each sweep step

Attachment Size

- bulk.tib 1.16 KB
- bulk.geo 181 bytes
- bulk.msh 4.49 KB

CHAPTER

FIVE

THERMAL

5.1 Introduction

The steady state heat flux, within the diffusive approach, is given by the Fourier?s law, i. e. $J_i = -\kappa_{ij} \frac{\partial T}{\partial x_j}$ where κ is thermal conductivity second-rank tensor (which is assumed temperature independent).

The power balance is ensured by the continuity equation for the thermal flux $\frac{\partial J_i}{\partial x_i} = H$ where H is the total heat source.

Thermal module computes the balance between the heat generated and the heat dissi pated. Here we get the temperature profile of a rectangular device with an hotspot in the center.

The device block specifies the material (Silicon) and the physical regions. In this case we have the HotSpot and the Base region

```
Device
  {
   material = Si
   Region HotSpot{}
   Region Base{}
  }
```

Let's now declare the thermal module. The temperature is fixed at the left and right side of the domain. The hotspot is included only in the HotSpot region

```
Module thermal
  {
   Physics
     {
      Contact left
        {
         type = heat_reservoir
         temperature = 300
        }
      Contact right
        {
         type = heat_reservoir
         temperature = 300
        }
      HeatSource constant
        {
         regions = HotSpot
         H = 1e10
        }
```

```
    }
  }
```

In the simulation region we write which modules should be solved.

```
Simulation{solve = thermal}
```

The temperature map is shown below

5.2 Boundary conditions

The boundaries of the thermal simulation domain are set to ideally thermal insulator by default.

Dirichlet boundary conditions can be imposed with the keyword heat_reservoir.

Example:

```
Contactheat_reservoir
  {
   region = substrate
   temperature = 300
  }
```

The surface resistance can be added with the keyword boundary surface_resistance. (r_surf is in cm2 W/ K).

Example:

```
Contactsubstrate
  {
   type = surface_resistance
   r_surf = 0. 05
   temperature = 300
  }
```

5.3 Heat sources

The constant value heat source can be included with heat_source constant. The heat source must be in W/cm3.

Example:

```
HeatSourceconstant
  {
   region = qdot
   H = 1e10
  }
```

Heat generated by the Joule?s effect is included with heat_source joule. The name of the drift-diffusion simulation is given by dd_simulation.

Example:

```
HeatSourcejoule
  {
   dd_simulation = dd
  }
```

CHAPTER

SIX

MODULE EFASCHROEDINGER

The EFA calculations are performed by the **Module** efaschroedinger A typical ex- ample is the following:

```
Module efaschroedinger
  {
   particle = el
   poisson_model_name = dd
   strain_model_name = strain # macrostrain
   name = quantum_el
   regions = quantum
   plot = (EigenFunctions, EigenEnergy, EnergyLevels)
   Solver
     {
      number_of_eigenstates = 10 # 30
     }
   Physics
     {
      model = conduction_band
     }
  }
```

6.1 Module options

The following options infiuence the behaviour of the Module efaschroedinger:

`particle = string` defines for which particle (electron or hole) Schroedinger equation is

solved Possible values are el and hl. A different Module efaschroedinger has to be defined for each particle to be solved.

`poisson_model_name = string` defines the name of the simulation (Module driftdiffu-

sion) that can provide electric potential

`strain_model_name = string` defines the name of the simulation (Module macrostrain)

that can provide elastic strain

`regions = string` defines the regions associated to this EFA simulation

6.2 Solver section

The Solver section of the Module efaschroedinger contains the following options:

`number_of_eigenstates = integer` defines the number of eigenvalues and eigenfunctions to be found.

`Dirichlet_bc_everywhere = boolean` : if true (default value), Dirichlet boundary

conditions are imposed over all the boundaries of the simulation region

`solver = string` : defines the solver for the eigenvalue problem, possible values are:

arnoldi, lapack, krylovshur. The default value is **krylovshur** . In the case of the lapack solver all the eigenvalues are computed. In the case of arnoldi or krylovshur solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, refered to as the *spectral_shift*

`max_iteration_number = integer` : maximum number of iteration, used as a stop condition

`eigen_solver_tolerance = double` :numerical eigensolver tolerance used as a convergence criteria

`spectral_shift = double` :the closest eigenvalues to this value (eV) are found. If not

defined, then it will be calculated internally from the band edges.

`ksp_type = string` : Krylov subspace method type; bcgsl, gmres, cg

`pc_type = string` : preconditioner type: cholesky, jacobi, ilu , composite.

6.3 Physics section

`model = string` : possible values are : conduction band , for single conduction band

model (Γ point) ; kp for $\mathbf{k} \cdot \mathbf{p}$ model

`kp_model = string` : possible values are : 6?6 , 8?8.

6.4 Output

The available output variables, to be specified in the plot option, are the following:

- EigenEnergy Eigen energy in eV
- EigenFunctions $|\psi(\mathbf{r})|^2$ function of the eigenstate
- Occupation probability to find the state occupied. It is calculated assuming Fermi

distribution and mean electrochemical potential and temperature:

- EnergyLevels graphical output used for showing the energy level over the band

diagram.

CHAPTER

SEVEN

MODULE OPTICSKP

By defining the Module **opticskp** , calculation of optical properties is enabled.

```
Module opticskp
  {
   name = optics
   regions = quantum
   plot = (optical_spectrum_k_0 )
   initial_state_model = quantum_el
   final_state_model = quantum_hl
   initial_eigenstates = (0, 9)
   final_eigenstates = (0, 15)
   polarization = (0, 0, 1)
   Emin = 2.8
   Emax = 3.6
   dE = 0.001
  }
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

7.1 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for *k=0*.

CHAPTER

EIGHT

MODULE OPTICALSPECTRUM

By defining the Module **opticalspectrum** , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
  {
   k_space_dimension = 2
   k-space_basis = true
   k1 = (0, 0, 0.1)
   k2 = (0, 0.1, 0)
   refine_fraction = 0.30
   relative_accuracy = 0.01
   refine_k_space = true
   number_of_nodes = (2, 2)
   wedge = quarter
   plot = (optical_spectrum)
   optical_matr_elem_model = opticskp
   polarization = (0, 0, 1)
   Emin = 3.0
   Emax = 5.0
   dE = 0.001
  }
```

The parameters are the following:

`k_space_dimension` = **1** for 2D simulations, **2** for 1D simulations. k-space basis is

true if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in

real space

If `refine_k_space` = **true** , that is adaptive k-mesh refinement is enabled, all the el-

ements whose error is greater than the value (1-refine fraction)* (maximum error) are going to be refined. In this case, ?Error? is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

`number_of_nodes` = numb. of elements in k mesh, along each direction

`wedge` = half | quarter, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin, Emax, dE` : energy range and step of spectrum calculation.

8.1 Output

The output variables for optics calculations are:

- **optical_spectrum** : k-space integrated optical emission spectrum.Chapter 5

CHAPTER

NINE

MODULE QUANTUMDISPERSION

With the Module quantumdispersion it is possible to calculate the dependence of quan- tum eigenstates on **k** -vector. Such dependence gives the *quantum state dispersion* . The simulation name is **quantumdispersion** .

```
Module quantumdispersion
  {
   simulation_name = dispersion1D_el
   regions = all
   quantum_simulation = quantum_el
   min_eigenvalue_number = 0
   max_eigenvalue_number = 5
   wedge = half
   k_space_dimension = 1
   k1 = (0, 0.1, 0)
   number_of_nodes = (10)
   output_format = grace
   plot = k-space_dispersion
  }
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

- **quantum simulation** : name of the efaschroedinger simulation.
- **min eigenvalue number** , **max eigenvalue number** : the dispersion is calculated

for the states number *i* , where

max_eigenvalue_number $\geq i \geq$ min_eigenvalue_number

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

9.1 Output

The output variable name is **k-space_dispersion** . The output format for the dispersion can be controlled independently of the general specification in the `Simulation` section by redefining the `output_format` keyword.

CHAPTER

TEN

MODULE QUANTUMDENSITY

In Module quantumdensity you can define the calculation of particle (electron,hole) den- sity, based on the result of a previous calculation of the system eigenstates (e.g. with efaschroedinger module).

```
Module quantumdensity
  {
   name = dens_el
   regions = quantum
   plot = quantum_density
   k-space_dimension = 0
   k-space_basis = true
   k1 = (0, 0, 0.1)
   #number_of_nodes = (4)
   #wedge = half
   quantum_simulation = quantum_el
   degeneracy = 2
   refine_fraction = 0.20
   relative_accuracy = 0.01
   refine_k_space = true
   output_density_in_k_space = true
   uniform_refinement = false
   mesh_order = FIRST
   first_state = 3
   initial_eigenstates_number = 10
   analytic = true
  }
```

The available options are:

- **quantum_simulation** : name of the efaschroedinger simulation.
- degeneracy: degeneracy of the quantum state
- **initial_eigenstates_number** : initial number of eigenstates for the Schroedinger

equation

- **analytic** = { true | false } If true then the density is calculated analytically,

otherwise numerically.

10.1 Output

The output parameter is **quantum_density**.

CHAPTER

ELEVEN

SIMULATION DYE SOLAR CELLS

11.1 Model DSC

The DSC model is tagged as *dssc* . In **options** subsection we indicate the simulation name:

```
model dssc
  {
   options
     {
      simulation_name = <name_of_the_model>
      plot(Potential, Density, Current, ContactCurrents)
     }
   BC_regions
     {
      ...
     }
  }
```

11.2 Boundary Conditions

The boundary conditions are inserted in the **Model** section, the two contacts are the photoanode and the cathode. The photoanode must be the boundary of a region where $\mathbf{TiO_2}$ is present, on the contrary the cathode must be the boundary of a region where the **electrolyte** is present.

```
BC_regions
  {
   BC_region <name_of_the_anode>
     {
      type = ohmic
      bias = @V[0.0]
     }
   BC_region <name_of_the_cathode>
     {
      type = Pt
      load = @R[1e8]
     }
  }
```

The anode is an ohmic contact, it contains only the sweep of the bias applied for the electrochemical potential of the electrons. The cathode must contain the initial external resistance (**load = R**). If a simulation of the cell under illumination is made the initial value of *load* must be set to a high value, in the range of $10^8 \omega cm^2$ in order to have no

current during the light intensity sweep. In case instead the simulation performed is under dark conditions where an external bias only is applied `load = 1.0` .

11.3 Generation model

The generation term is related to the flux of photons which reaches the active TiO_2 regions and the dye present in the cell. We assume a simple Beer-Lambert exponential decay for charge generation of the form:

$$G(\mathbf{r}) = \int_{\lambda_1}^{\lambda_2} \alpha(\lambda)\Phi(\lambda)e^{-\alpha(\lambda)\hat{n}\cdot\mathbf{r}}d\lambda \tag{11.1}$$

where α is the absorption coeflcient (in μ^{-1}) of the chosen Dye, $\Phi(\lambda)$ the intensity of the light power at that wavelength for the spectrum of the sun. The part of the input flle for the generation model:

```
model dssc_generation
  {
   options
     {
      regions = (<TiO2_region_name_1>, <TiO2_region_name_2>, ...)
      plot = (Distance, Generation)
      light_direction = <vector_indicating_the_direction_of_light>
     (example, illumination from x direction: light_direction = (1, 0, 0))
      light_intensity = @x
      dye = N719
     }
   BC_Regions
     {
      BC_Region <name_of_the_boundary>
        {
        }
     }
  }
```

In the generation input flle must be specifled:

- (**regions**) the regions where we want that there is generation (where the Dye is present);
- (**plot**) if we want to plot the generation;
- (**light_direction**) the vector which flxes the direction from where the light comes;
- (**light_intensity**) the light intensity;
- (**light_intensity**) the light intensity;
- (**dye**) the dye used in the cell;

The light intensity is deflned in a sweep (see section *Solver*). It can be set to reach 1 that means one Sun, or a larger or smaller illumination intensity (0.1, 2.0, etc.). There is another flag called **illumination_spectrum** that can be used if we want to change the spectrum proflle F with another illumination source (for example if we assume the cell under concentration of light). The flle used by default for Φ is the standard 1.5 AM spectrum of the sun contained in the material database in the flle `Sun1p5am` .

The last part of the generation is the deflnition of the boundary. This boundary says to the code which is the boundary region of the grid from where the light penetrates in the device. The information given by the boundary and the light direction vector deflnes the coming of light and direction of rays.

11.3.1 Device

Device section for a DSC simulation:

```
Device
  {
   Region <name of the porous region>
     {
      mat = TiO2mes
      porosity = 0.5
     }
   Region <name of the electrolyte region>
     {
      mat = TiO2mes
      TiO2 = false
     }
  }
```

For every region of the device it is specifled the material flle from the database (the TiO2mes contains standard parameters for both TiO_2 and electrolyte, so it can be used for both regions). For every region must be specifled if it contains TiO_2, or electrolyte or both. This can be done setting two flags called `TiO2` and `electrolyte` . If set true the material is present in the region, if set **false** it is not present. By default they are assumed both **true** (porous region). In the second region of the example showed there is electrolyte only and `TiO2` = **false** is explicitly specifled. In case both materials are present (porous region) a porosity must be specifled (in the range between 0, TiO_2 only, and 1, electrolyte only). If one material is not present the porosity is automatically set to 0 or 1.

11.3.2 Physics

The set of parameters that can be deflned for the entire device are enlisted in table 3.1. For the generation:

```
dssc
  {
   generation = dssc_generation
  }
```

11.3.3 Solver

Three sweeps are needed for the plot of the entire I-V under illumiantion. The flrst sweep is needed to make the transition from dark conditions to full open-circuit conditions under

illumination. The high value of the load R maintains the current to zero. Then a second sweep is used to pass from open circuit condition to short circuit condition lowering the external load from a high external load to a small one (R = 1). Finally, the voltage sweep compute the I-V characteristics under illumination. In case of dark simulation (application of an external bias without illumination) the flrst and second sweep are not needed. The external load for the cathode can be set directly to R = 1.

```
Solver
  {
   Sweep
     {
      sweep_gen
        {
         simulation = (dssc_generation, dssc)
         variable = x
         values = (0, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
         plot_data = true
```

Parameters		
	Poisson equation	
porosity	Porosity of porous material	
perm_oxide	Relative perm. of the TiO_2	
perm_electrolyte	Relative perm. of the electrolyte	
k_3	Oxidized Dye regeneration rate constant	s^{-1}
	Drift Diffusion equation	
ne	Dark electron density	cm^{-3}
nI	Dark iodide density	mol/l
nI3	Dark electron density	mol/l
mu_e	Electron mobility	cm^2/Vs
D_I	Iodide diffusion constant	cm^2/s
D_I3	Triiodide diffusion constant	cm^2/s
D_C	Cation diffusion constant	cm^2/s
	Recombination term	
k_e	Recombination constant rate	s^{-1}
beta	Non-linearity exponent in the electron density recombination	

Table 11.1: Physical parameters that can be set for the model divided in subsets relative to different processes in the cell.

```
    }
  sweep_R
    {
     simulation = dssc
     variable = R
     values = ( 1000, 100, 1)
     plot_data = true
     }
  sweep_V
    {
     simulation = dssc
     variable = V
     values = (0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0,8, 0.9, 1.0)
     plot_data = true
    }
  }
}
```

The intensity of illumination can be changed sweeping the value of x different to 1 (1 = 1 Sun of power).

11.3.4 Output

The output that want to be plotted are enlisted in section Model within option. See *Table nodal* , *Table elemental* and *Table scalar* .

Nodal quantities

	`Potential`	
`eQFermi`	Electron electrochemical potential	eV ($-e\phi_n$)
`IQFermi`	Iodide electrochemical potential	eV ($-e\phi_{I^-}$)
`I3QFermi`	Triiodide electrochemical potential	eV ($-e\phi_{I_3^-}$)
`CQFermi`	Cation electrochemical potential	eV ($-e\phi_C$)
`ElPotential`	Electrostatic potential	eV
`Eredox`	Electrolyte electrochemical potential	eV ($-e\phi_{Red}$)
	`Density`	
`eDensity`	Electron density	cm^{-3}
`IDensity`	Iodide density	cm^{-3}
`I3Density`	Triiodide density	cm^{-3}
`CDensity`	Cation density	cm^{-3}
`Generation`	The net electron generation rate	$cm^{-3}s^{-1}$
`NetRecombination`	The net recombination rate	$cm^{-3}s^{-1}$

Table 11.2: Nodal quantities. The flags `Potential` and `Density` allow to plot all the electrochemical potentials and all the densities, including generation and recombination.

Elemental quantities

	`Current`	
`ElField`	Electric Field	Vcm^{-1}
`CurrentDensity`	Total current density	Acm^{-2}
`eCurrentDensity`	Electron current density	Acm^{-2}
`ICurrentDensity`	Iodide current density	Acm^{-2}
`I3CurrentDensity`	Triiodide current density	Acm^{-2}
`CCurrentDensity`	Cation current density	Acm^{-2}

Table 11.3: Elemental quantities. The flag `Current` allows to plot all of them in the x, y and z components.

Scalar quantities

`ContactCurrents`	Contact currents	*[a]

[a] depends on dimension and symmetry

Table 11.4: Scalar quantities.

CHAPTER

TWELVE

GLOSSARY

modules This is an example of the Syntax view

See *modules* for an overview over Modules option.

constant charge a constant charge can be assigned by specifying only the sheet carrier density Ns in cm^{-2}. The sheet charge density will then equal Ns multiplied by the elementary charge e. A positive Ns produces a positive surface charge.

electronic surface states in this case the surface charge is produced by electrons oc- cupying a surface state with a density of states in form of a delta function. The density of occupied states then reads

degeneracy degeneracy of the quantum state

BIBLIOGRAPHY

[Kalyanasundaram] Kalyanasundaram Kuppuswamy, “Dye-Sensitized Solar Cells”, *Editor Crc Pr Inc.* , 2010.

[Gagliardi] Alessio Gagliardi, Matthias Auf der Maur, Desiree Gentilini and Aldo Di Carlo, “Modeling of Dye sensitized solar cells using a finite element method”, *J. Computational Electronics* , vol. 8, pp. 12, 2009.

[Wachutka] Wachutka, IEEE Trans CAD Cicui. Integr and Syst. (1990)].

[Povolotskiy] Povolotskiy et al. JAP (2006).

[Wang] Wang et al. Science (2006).

[Gao] Gao et al NanoLetter (2009).

INDEX

C

constant charge, 43

D

degeneracy, 43

E

electronic surface states, 43

M

modules, 43